

25/05/2025

**Sous l'encadrement : MR COSTA**

# PROJET FINAL ROBOTIQUE

[Q2] Robotique part 2 [A2073] [2024]



TCHOULA CARDOT HE304737  
DONFACK MAFOZAP BEBA CARELLE HE304882  
CALIK HANDE HE304998  
UWIZEYE ROSINE HE305004

# Table des matières

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>I.Introduction.....</b>                                                 | <b>3</b>  |
| <b>II.Partie électronique et conclusion partielle .....</b>                | <b>6</b>  |
| 1.Implementation logicielle pour le pilotage de la carte de commande ..... | 12        |
| 2.Materiel utilisé .....                                                   | 14        |
| 3.Conclusion partielle de la partie electronique .....                     | 17        |
| <b>III.Partie mécanique et conclusion parfaite .....</b>                   | <b>19</b> |
| 1.Conception de la partie mobile sur fusion 360 .....                      | 22        |
| 2.Conception de la manette sur fusion 360 .....                            | 24        |
| 3.Conclusion partielle de la partie mécanique.....                         | 27        |
| <b>IV.Partie programmation et conclusion partielle .....</b>               | <b>30</b> |
| 1.Code de la partie mobile .....                                           | 34        |
| 2.Code de la partie commande commande.....                                 | 37        |
| 3.Conclusion partielle de la partie logiciel .....                         | 42        |
| <b>V.Difficultés rencontrées .....</b>                                     | <b>47</b> |
| <b>VI.Conclusion générale .....</b>                                        | <b>53</b> |
| <b>VII.Bibliographie .....</b>                                             | <b>58</b> |
| <b>VIII.Algorigramme .....</b>                                             | <b>63</b> |
| <b>XI.Annexe .....</b>                                                     | <b>68</b> |

# I.INTRODUCTION

Ce projet de robotique s'inscrit dans le cadre de la révolution technologique actuelle, qui vise à simplifier et améliorer nos tâches quotidiennes. L'idée principale est de concevoir un robot capable de fonctionner selon deux modes : un mode autonome et un mode manuel.

En mode autonome, le robot utilise des capteurs et un programme de navigation pour se déplacer tout seul, éviter les obstacles et accomplir certaines tâches sans que quelqu'un le contrôle.

En mode manuel, l'utilisateur peut piloter le robot à l'aide d'une manette connectée en Bluetooth. Ce mode est utile dans les situations où une intervention humaine est nécessaire, comme pour des actions précises ou imprévues.

Ce robot sera donc polyvalent, capable de s'adapter à différents besoins. Il pourra par exemple travailler de manière automatique dans des environnements simples, mais aussi être contrôlé à distance quand il faut plus de précision.

Pour réaliser ce projet, nous allons utiliser des éléments de mécanique, d'électronique, de programmation et de communication sans fil. Le but est de construire un robot fiable, efficace et flexible, capable de s'adapter aux situations qu'il rencontre.

## II.PARTI ELECTRONIQUE

### II.1)Implémentation logicielle pour le pilotage de la carte de commande

Dans cette seconde phase de conception, nous avons réalisé le câblage de la section dédiée à la commande du robot. Cette partie permet un contrôle manuel à distance à l'aide d'une manette équipée des composants suivants :

- Un module Bluetooth HC-05 : Il assure la communication sans fil entre la manette et le robot, permettant de contrôler la partie mobile à distance.
- Une LED RGB : Elle indique le mode de fonctionnement sélectionné en affichant différentes couleurs (rouge, bleu, vert).
- Une LED verte : Elle signale l'état de la connexion Bluetooth, permettant de vérifier en un coup d'œil si la communication est bien établie.
- Un joystick : Il permet de diriger le robot vers l'avant, l'arrière, la gauche ou la droite.
- Des boutons-poussoirs : Ils servent à activer ou désactiver la fonctionnalité de balayage du bras robotisé.

Après de minutieuses vérifications, nous avons procédé à la commande des premiers PCB. La version physique du circuit imprimé correspondant à la partie mobile se présente comme suit:

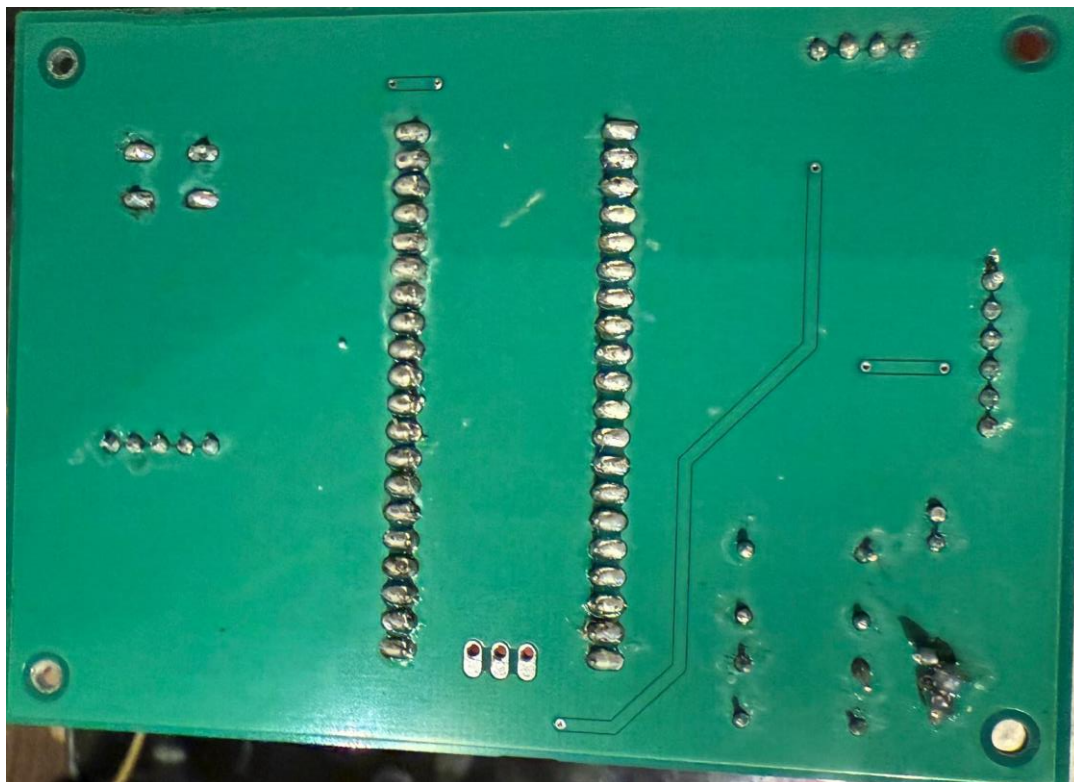
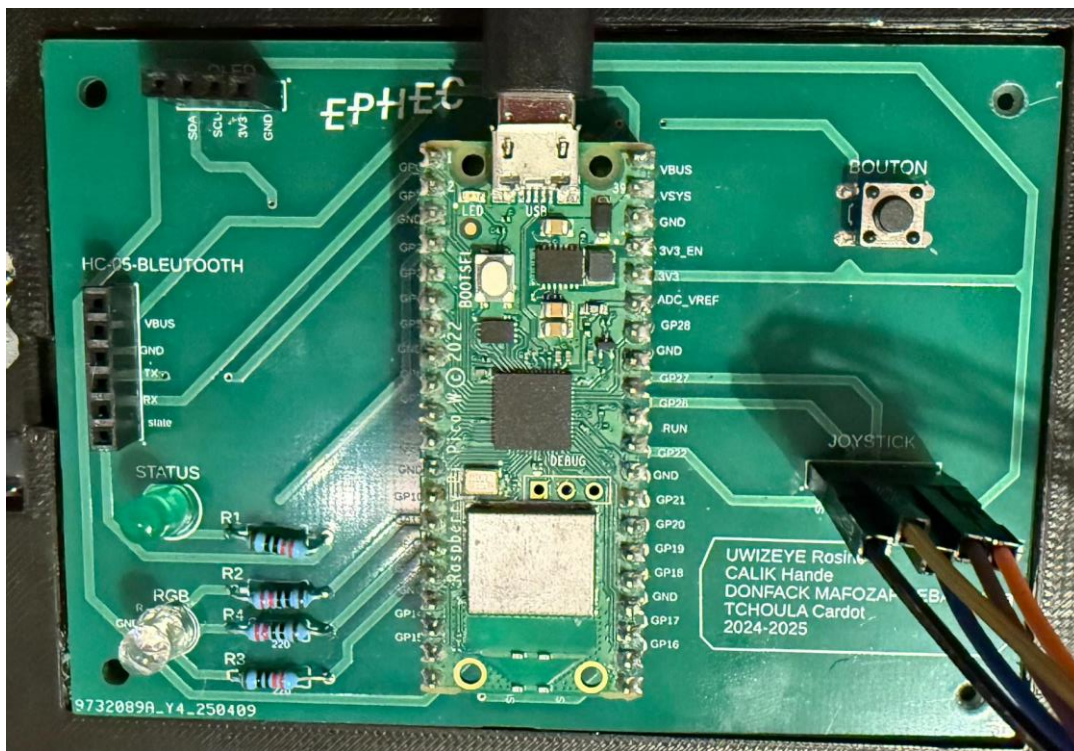


Figure 1 : Conception logicielle PCB point de vue commande :

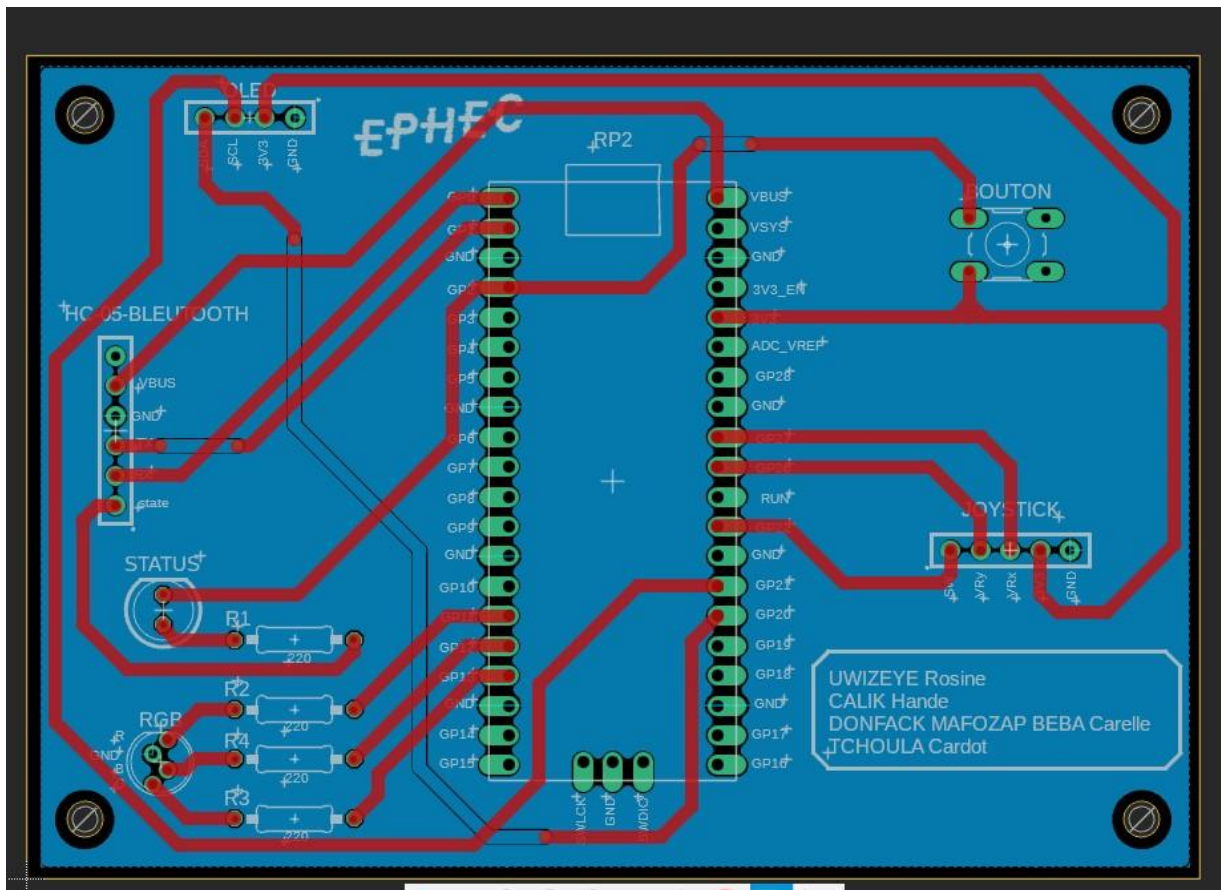
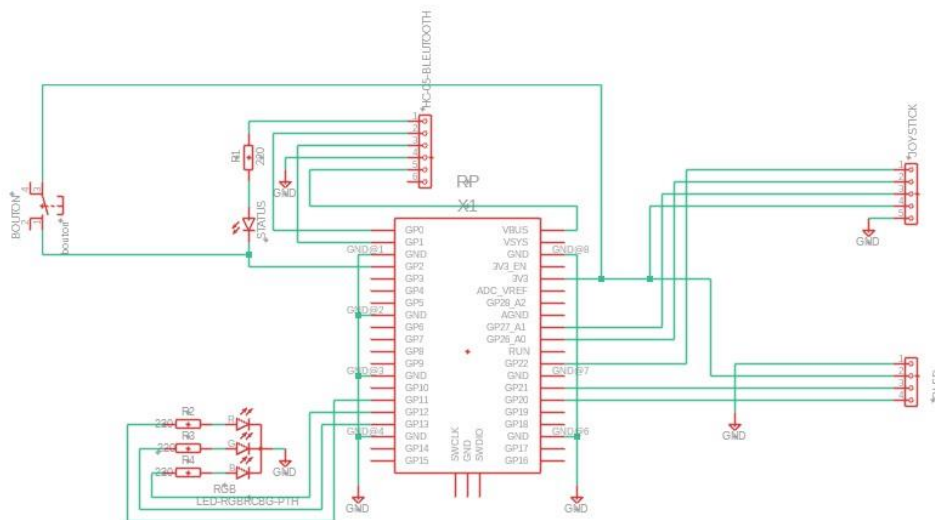


Figure 9 : Routage PCB point de vue commande :



## II.2) Matériel utilisé

### Manette:



-Bouton poussoir

-Oled

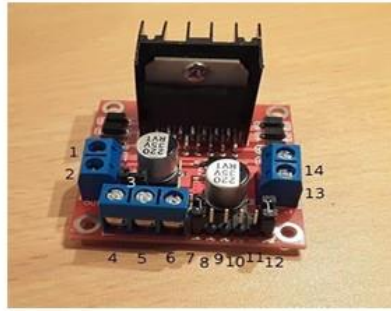
- Joystick

-Rgb

-Hc05

-Led vert

### Robot :



Capteur ultrason

HC02



LED RGB



- Bouton poussoir
- Oled
- I295n
- Rgb
- Hc05
- Ultra son
- Servo moteur
- Led vert
- Rgb

## II.3) Conclusion partielle de la parti electronique

La partie électronique de notre projet a été soigneusement étudiée afin d'intégrer l'ensemble des fonctionnalités nécessaires au pilotage et au contrôle du robot, aussi bien en mode manuel qu'en mode autonome. Nous avons procédé au câblage de tous les composants essentiels, notamment :

- Le module Bluetooth HC-05, utilisé pour assurer la communication sans fil avec la manette.
- Un servomoteur, destiné à la manipulation d'obstacles ou à l'actionnement du bras robotisé.
- Un capteur à ultrasons, permettant la détection d'obstacles lors des déplacements autonomes.
- Un écran OLED, pour l'affichage d'informations telles que les directions et les messages d'état.
- Une LED RGB, servant de signalisation visuelle selon les différents modes.
- Un joysticks, pour la navigation manuelle du robot.
- L'ensemble des composants a été alimenté en 3V et 5V selon leurs spécifications respectives, afin de garantir un fonctionnement optimal lors des phases de test et d'utilisation.

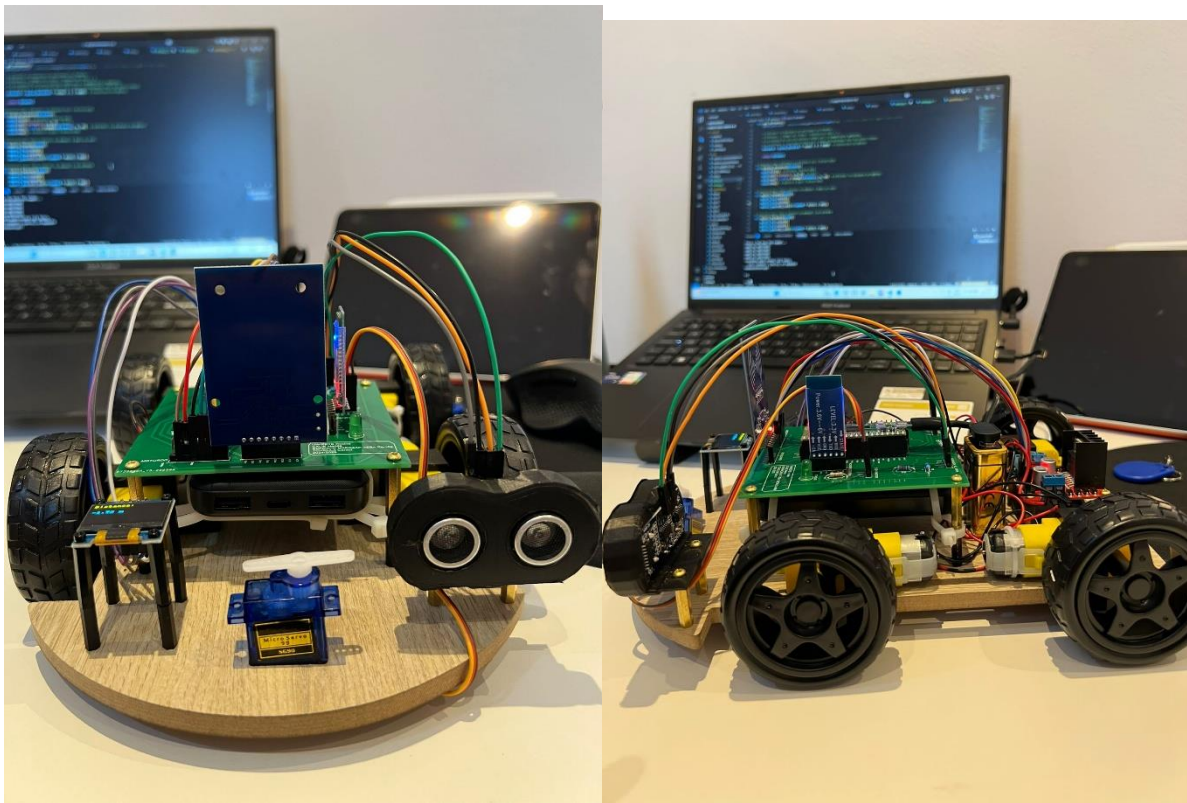
Enfin, une alimentation par batterie a été prévue pour permettre un fonctionnement totalement autonome du robot, sans dépendance à une source d'alimentation externe. Cette configuration électronique permet une communication fluide entre les modules et assure la stabilité et la performance du système dans divers scénarios d'application.

## III.PARTI MECANIQUE

### III.1)Conception de la partie mobile sur Fusion 360

Dans le cadre de notre projet de véhicule télécommandé, nous avons entrepris la modélisation 3D de la partie mobile du châssis ainsi que de la manette de contrôle, en utilisant le logiciel **Fusion 360**. Cette étape préliminaire nous a permis de visualiser l'architecture globale du système avant sa réalisation physique.

La première version du châssis intègre les composants essentiels au fonctionnement du véhicule : moteur, batterie, microcontrôleur, et modules de transmission. Nous avons conçu des ouvertures stratégiquement positionnées pour faciliter le passage des câbles, assurant ainsi une gestion optimale du câblage et une maintenance aisée.



### III.2) Conception de la manette sur Fusion 360

Dans le cadre de notre projet de véhicule télécommandé, nous avons conçu une manette ergonomique permettant de contrôler la voiture à distance. Pour cela, nous avons utilisé le logiciel **Fusion 360** afin de modéliser en 3D l'ensemble des composants et de réaliser une impression 3D précise de la structure.

La manette intègre les éléments suivants :

- Un joystick analogique : permettant le contrôle directionnel du véhicule.
- Un module Bluetooth HC-05 : assurant la communication sans fil entre la manette et la voiture.
- Un écran OLED de 0,96 pouce : affichant les informations de direction et les modes de fonctionnement.
- Une LED RGB : indiquant visuellement les modes de fonctionnement grâce à des codes couleur spécifiques.
- Un interrupteur : servant à contrôler activation et la désactivation du bras robotiser.
- Une led verte servant à visualise s'il y'a communication ou pas.

Figure : Conception logicielle de la manette



Après avoir finalisé la modélisation 3D de notre manette à l'aide de Fusion 360, nous avons procédé à son impression en utilisant une imprimante 3D. Ce processus nous a permis de concrétiser notre conception virtuelle en un prototype physique fonctionnel.

Figure : Manette

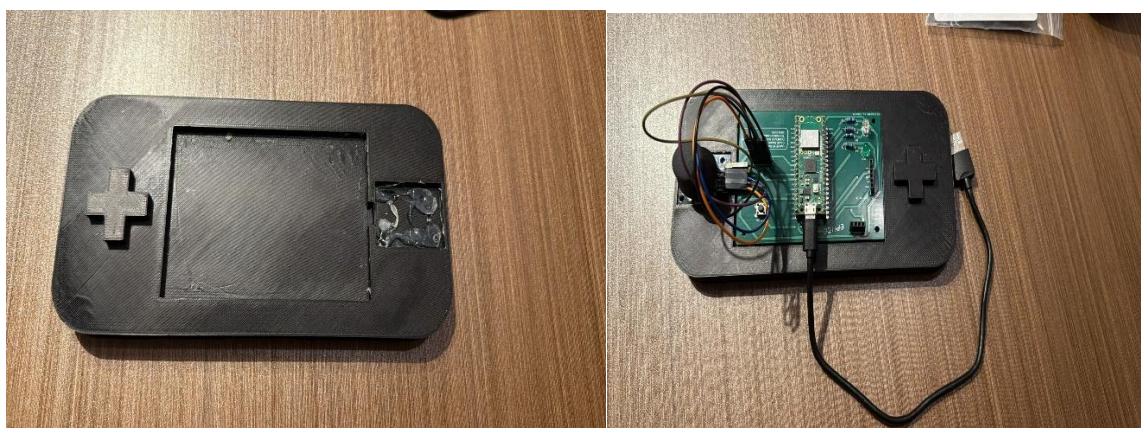
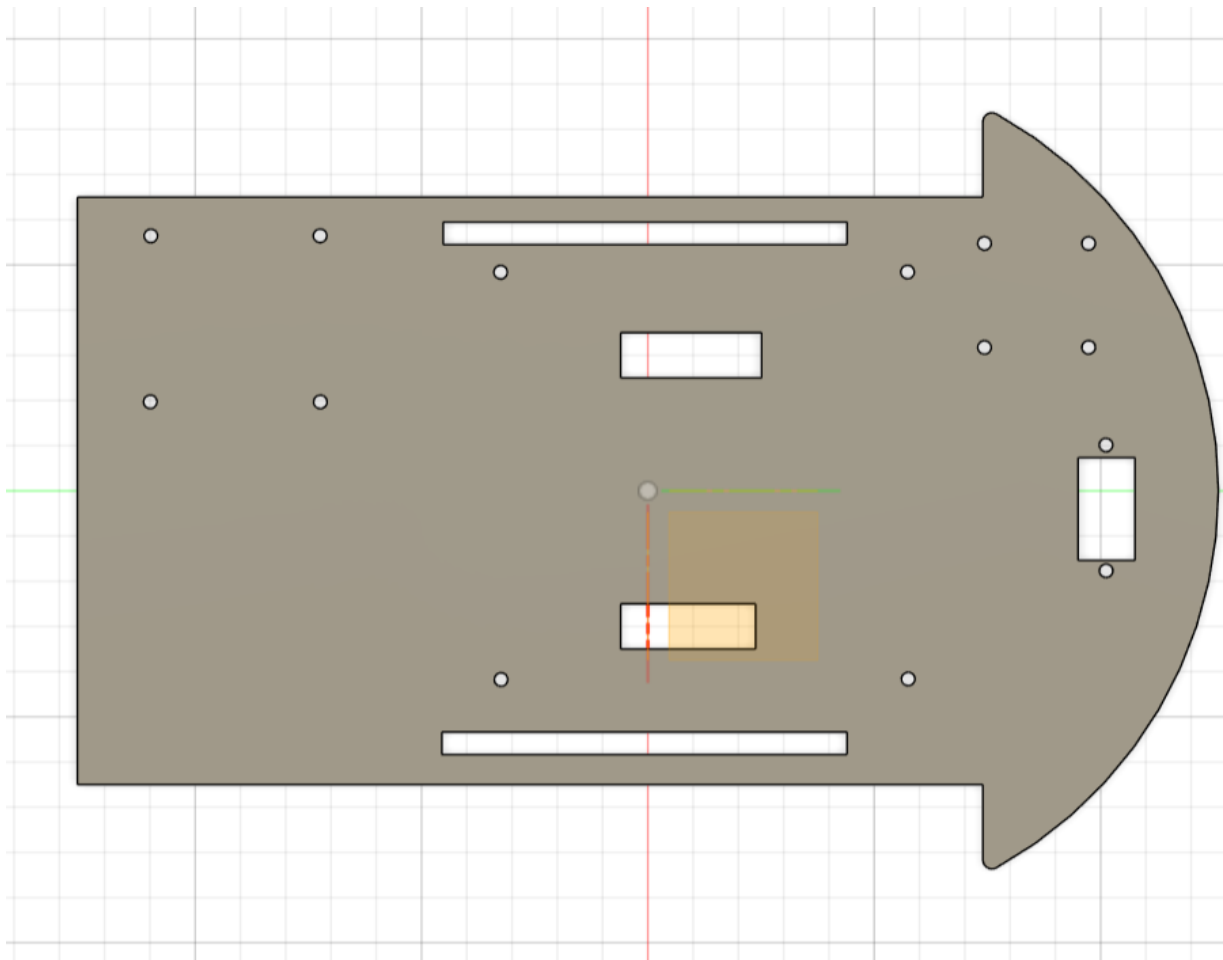




Figure : Conception logicielle du châssis



La partie mécanique de notre projet a été soigneusement élaborée afin de garantir une intégration optimale et fonctionnelle de l'ensemble des composants électroniques. Grâce au logiciel **Fusion 360**, nous avons pu modéliser en 3D le châssis de la manette ainsi que celui du robot. Cette modélisation nous a permis de visualiser avec précision l'agencement des composants et de valider leur compatibilité avant toute phase de fabrication.

Nous avons intégré :

- Des **logements dédiés** pour chaque composant (joysticks, module Bluetooth, écran OLED, LED RGB, interrupteur),
- Des **perçages stratégiquement positionnés** pour le passage et la gestion des câbles,
- Des **soutiens mécaniques renforcés** assurant la stabilité et la protection des modules.

La visualisation 3D nous a permis d'anticiper d'éventuels problèmes d'encombrement, d'optimiser l'ergonomie et d'ajuster les dimensions afin d'obtenir un assemblage précis. Cette démarche a contribué à une fabrication efficace, facilitant l'intégration des parties électronique et mécanique pour un fonctionnement fluide et cohérent de l'ensemble du système robotisé.

### III.3) Conclusion partielle de la parti mécanique

la partie mécanique de notre projet a été soigneusement élaborée afin de garantir une intégration optimale et fonctionnelle de l'ensemble des composants électroniques. Grâce au logiciel **Fusion 360**, nous avons pu modéliser en 3D le châssis de la manette ainsi que celui du robot. Cette modélisation nous a permis de visualiser avec précision l'agencement des composants et de valider leur compatibilité avant toute phase de fabrication.

Nous avons intégré :

- Des **logements dédiés** pour chaque composant (joysticks, module Bluetooth, écran OLED, LED RGB, interrupteur),
- Des **perçages stratégiquement positionnés** pour le passage et la gestion des câbles,
- Des **supports mécaniques renforcés** assurant la stabilité et la protection des modules.

La visualisation 3D nous a permis d'anticiper d'éventuels problèmes d'encombrement, d'optimiser l'ergonomie et d'ajuster les dimensions afin d'obtenir un assemblage précis. Cette démarche a contribué à une fabrication efficace, facilitant l'intégration des parties électronique et mécanique pour un fonctionnement fluide et cohérent de l'ensemble du système robotisé.

# IV.PARTI LOGICIEL

## IV.1) Code de la partie mobile

```
1 # Importation des modules nécessaires pour la communication, le contrôle des moteurs, l'affichage,
2 etc.
3 from machine import UART, Pin, PWM, time_pulse_us, I2C, SPI
4 import time
5 import ssd1306
6 from mfrc522 import MFRC522
7 from motorMA import ManuController
8
9 # Initialisation du port UART pour la communication Bluetooth
10 uart = UART(0, baudrate=38400, tx=Pin(0), rx=Pin(1))
11
12 # Initialisation de la classe de contrôle manuel des moteurs
13 motorMA = ManuController()
14
15 # Configuration de l'afficheur OLED via I2C
16 i2c = I2C(0, scl=Pin(21), sda=Pin(20))
17 oled = ssd1306.SSD1306_I2C(128, 64, i2c)
18
19 # Initialisation des broches pour LED RGB
20 led_red = Pin(11, Pin.OUT)
21 led_green = Pin(13, Pin.OUT)
22 led_blue = Pin(12, Pin.OUT)
23
24 # Fonction pour définir la couleur de la LED RGB
25 def led_color(r, g, b):
26     led_red.value(1 if r else 0)
27     led_green.value(1 if g else 0)
28     led_blue.value(1 if b else 0)
29
30 # Limite de fréquence de mise à jour de l'écran OLED
31 last_oled_update = 0
32
33 # Mise à jour de l'écran OLED avec l'état actuel
34 def update_oled_status(mode, direction):
35     global last_oled_update
36     now = time.ticks_ms()
37     if time.ticks_diff(now, last_oled_update) < 150:
38         return
39     last_oled_update = now
40
41     oled.fill(0)
42     oled.text(" VOITURE DEBLOQUEE", 10, 0)
43     oled.text("Mode:", 0, 15)
44     if mode == "AUTO":
45         oled.text("AUTO", 45, 15)
46         oled.line(45, 25, 80, 25, 1)
47     else:
48         oled.text("MANU", 45, 15)
49         oled.line(45, 25, 85, 25, 1)
50     oled.text("sens: " + direction, 0, 40)
51     oled.show()
52
53 # Configuration du lecteur RFID via SPI
54 spi = SPI(0, baudrate=1000000, polarity=0, phase=0,
55           sck=Pin(18), mosi=19, miso=16)
56 rdr = MFRC522(spi_id=0, sck=18, mosi=19, miso=16, cs=17, rst=10)
57
```

```

58 # Liste des identifiants de badge autorisés
59 authorized_uids = [[760619171]]
60
61 # Configuration du servo moteur
62 servo = PWM(Pin(28))
63 servo.freq(50)
64
65 # Fonction pour régler l'angle du servo
66 def set_servo_angle(angle):
67     duty = int(1638 + (angle / 90.0) * 4915)
68     servo.duty_u16(duty)
69
70 # Configuration du capteur ultrason (TRIG et ECHO)
71 TRIG = Pin(4, Pin.OUT)
72 ECHO = Pin(5, Pin.IN)
73
74 # Fonction pour mesurer la distance en cm avec l'ultrason
75 def distance_cm():
76     TRIG.low()
77     time.sleep_us(2)
78     TRIG.high()
79     time.sleep_us(10)
80     TRIG.low()
81     try:
82         duration = time_pulse_us(ECHO, 1, 30000)
83         return (duration / 2) * 0.0343
84     except:
85         return 999
86
87 # Variables pour faire osciller le servo moteur (balayage)
88 servo_angle = 0
89 servo_direction = 1
90
91 # Fonction de balayage du servo de 0 à 90 degrés
92 def balayage_servo():
93     global servo_angle, servo_direction
94     servo_angle += 5 * servo_direction
95     if servo_angle >= 90:
96         servo_angle = 90
97         servo_direction = -1
98     elif servo_angle <= 0:
99         servo_angle = 0
100         servo_direction = 1
101     set_servo_angle(servo_angle)
102
103 # Définition du seuil de détection d'obstacle
104 SEUIL_CM = 20
105 obstacle_detected = False
106
107 # État initial affiché sur OLED en attente de badge
108 oled.fill(0)
109 oled.text("VOITURE DEBLOQUER ", 20, 0)
110 oled.text("ID: Waiting", 5, 30)
111 oled.show()
112
113 # Attente de la présentation d'un badge RFID valide
114 badge_valide = False
115 led_color(0, 0, 1) # LED bleue allumée
116
117 while not badge_valide:
118     (stat, tag_type) = rdr.request(rdr.REQIDL)
119     if stat == rdr.OK:
120         (stat, raw_uid) = rdr.anticoll(0x93)
121         if stat == rdr.OK:
122             if raw_uid in authorized_uids:

```

```

123         badge_valide = True
124         uart.write("lock:0\n")
125         led_color(1, 0, 0) # LED rouge
126         oled.fill(0)
127         oled.text("ID: Access :", 0, 28)
128         oled.text("Card Unlocked", 20, 0)
129         oled.show()
130         time.sleep(1)
131     else:
132         led_color(0, 0, 1)
133         oled.fill(0)
134         oled.text("ID: Not Access", 0, 28)
135         oled.text("Card Locked", 20, 0)
136         oled.show()
137         time.sleep(2)
138         oled.fill(0)
139         oled.text("ID: Waiting", 0, 28)
140         oled.text("Card Locked", 20, 0)
141         oled.show()
142         led_color(0, 1, 0) # LED verte
143     time.sleep(0.2)
144
145 # Boucle principale du programme après identification réussie
146 buffer = ""
147 last_check = time.ticks_ms()
148 auto_mode = False # Mode manuel par défaut
149
150 while True:
151     # Vérifie la présence d'obstacle toutes les 50 ms
152     if time.ticks_diff(time.ticks_ms(), last_check) > 50:
153         distance = distance_cm()
154         obstacle_detected = distance < SEUIL_CM
155
156         if auto_mode:
157             if not obstacle_detected:
158                 motorMA.move_backward(30000, 30000)
159                 update_oled_status("AUTO", "ROULE")
160             else:
161                 motorMA.stop()
162                 update_oled_status("AUTO", "Obstacle")
163         else:
164             if not obstacle_detected:
165                 balayage_servo()
166             last_check = time.ticks_ms()
167
168     # Lecture de données depuis Bluetooth UART
169     if uart.any():
170         char = uart.read(1).decode()
171         if char == '\n':
172             try:
173                 direction, value = buffer.strip().split(":")
174                 value = int(value)
175
176                 if direction == "auto":
177                     auto_mode = (value == 1)
178                     uart.write(f"auto:{value}\n")
179                     update_oled_status("AUTO" if auto_mode else "MANU", "Standby")
180                     if not auto_mode:
181                         motorMA.stop()
182
183                 elif not auto_mode:
184                     if direction == "forward":
185                         motorMA.move_forward(value, value)
186                         update_oled_status("MANU", "Forward")
187                     elif direction == "backward":

```

```

188             motorMA.move_backward(value, value)
189             update_oled_status("MANU", "Backward")
190         elif direction == "left":
191             motorMA.turn_left(value, value)
192             update_oled_status("MANU", "Left")
193         elif direction == "right":
194             motorMA.turn_right(value, value)
195             update_oled_status("MANU", "Right")
196         elif direction == "stop":
197             motorMA.stop()
198             update_oled_status("MANU", "Stop")
199     except Exception as e:
200         print("Erreur UART:", buffer, "| Exception:", e)
201         buffer = ""
202     else:
203         buffer += char

```

## EXPLICATION DU CODE

### 1. Importation des modules

```

from machine import UART, Pin, PWM, time_pulse_us, I2C, SPI
import time
import ssd1306
from mfrc522 import MFRC522
from motorMA import ManuController

```

Ce bloc de code permet de charger tous les modules nécessaires au bon fonctionnement du robot. Le module machine fournit les outils pour accéder directement aux fonctions matérielles de la carte comme la communication série, les broches numériques, les signaux PWM ou encore les interfaces de communication I2C et SPI. Le module time permet de gérer les délais, les horloges et les temporisations nécessaires pour coordonner les actions du robot. Le module ssd1306 est utilisé pour piloter un écran OLED connecté via I2C, tandis que MFRC522 permet de gérer un lecteur de badge RFID fonctionnant avec le protocole SPI. Enfin, ManuController est une classe externe personnalisée qui regroupe les fonctions permettant de commander les moteurs de la voiture en mode manuel, elle est probablement définie dans un fichier annexe appelé motorMA.py.

### 2. Initialisation des périphériques

```

# Initialisation du port UART pour la communication Bluetooth
uart = UART(0, baudrate=38400, tx=Pin(0), rx=Pin(1))

# Initialisation de la classe de contrôle manuel des moteurs
motorMA = ManuController()

# Configuration de l'afficheur OLED via I2C
i2c = I2C(0, scl=Pin(21), sda=Pin(20))
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

```

Ici, le programme initialise les interfaces principales de communication et d'affichage. Le port UART numéro 0 est configuré pour la communication Bluetooth avec un débit de 38400 bauds, ce qui permet d'échanger des instructions entre un appareil externe comme un smartphone et le microcontrôleur. L'objet motorMA est une instance de la classe ManuController, il va servir à déclencher les déplacements du robot dans différentes directions en mode manuel. Ensuite, le bus I2C numéro 0 est activé avec les broches 21 pour SCL (horloge) et 20 pour SDA (données), il est utilisé pour faire fonctionner un petit écran OLED de résolution 128x64 pixels qui permet d'afficher en temps réel des informations comme l'état de la voiture, son mode ou encore les messages liés à l'accès par badge.

### 3. Initialisation des broches pour LED RGB

```
# Initialisation des broches pour LED RGB
led_red = Pin(11, Pin.OUT)
led_green = Pin(13, Pin.OUT)
led_blue = Pin(12, Pin.OUT)

# Fonction pour définir la couleur de la LED RGB
def led_color(r, g, b):
    led_red.value(1 if r else 0)
    led_green.value(1 if g else 0)
    led_blue.value(1 if b else 0)
```

Dans ce bloc, trois broches numériques sont configurées en sortie pour piloter les trois couleurs de base d'une LED RGB, à savoir le rouge sur la broche 11, le vert sur la broche 13 et le bleu sur la broche 12. Une fonction nommée `led_color` est définie pour simplifier l'allumage de ces couleurs. Cette fonction prend en paramètre trois valeurs booléennes correspondant aux composantes rouge, verte et bleue, et active ou désactive chaque couleur en écrivant une valeur haute ou basse sur les broches correspondantes, ce qui permet d'indiquer visuellement l'état du système comme l'attente d'un badge, un accès refusé ou autorisé.

#### 4. Gestion de l'affichage OLED

```
# Limite de fréquence de mise à jour de l'écran OLED
last_oled_update = 0

# Mise à jour de l'écran OLED avec l'état actuel
def update_oled_status(mode, direction):
    global last_oled_update
    now = time.ticks_ms()
    if time.ticks_diff(now, last_oled_update) < 150:
        return
    last_oled_update = now

    oled.fill(0)
    oled.text(" VOITURE DEBLOQUEE", 10, 0)
    oled.text("Mode:", 0, 15)
    if mode == "AUTO":
        oled.text("AUTO", 45, 15)
        oled.line(45, 25, 80, 25, 1)
    else:
        oled.text("MANU", 45, 15)
        oled.line(45, 25, 85, 25, 1)
    oled.text("sens: " + direction, 0, 40)
    oled.show()
```

Ce segment initialise une variable pour limiter la fréquence de mise à jour de l'écran OLED, évitant ainsi des rafraîchissements trop rapides qui pourraient ralentir le système. La fonction `update_oled_status` est ensuite définie pour afficher l'état de la voiture. Elle prend deux arguments, le mode de fonctionnement (manuel ou automatique) et la direction actuelle de déplacement, comme avant ou arrière. Si la dernière mise à jour de l'écran est récente (moins de 150 millisecondes), la fonction ne fait rien pour ne pas saturer l'écran. Sinon, elle efface le contenu de l'écran, affiche l'état actuel de la voiture, indique le mode sélectionné avec un soulignement graphique, et précise la direction active, ce qui offre une visibilité claire à l'utilisateur sur le fonctionnement du robot.

## 5. Initialisation du lecteur RFID

```
# Configuration du lecteur RFID via SPI
spi = SPI(0, baudrate=1000000, polarity=0, phase=0,
          sck=Pin(18), mosi=19, miso=16)
rdr = MFRC522(spi_id=0, sck=18, mosi=19, miso=16, cs=17, rst=10)

# Liste des identifiants de badge autorisés
authorized_uids = [[760619171]]
```

Ici, le lecteur RFID est configuré pour fonctionner avec le protocole SPI, en utilisant les broches spécifiées pour l'horloge, les données entrantes et sortantes, ainsi que les broches de sélection et de réinitialisation. Un objet rdr de type MFRC522 est créé pour gérer les interactions avec les badges RFID. Une liste appelée `authorized_uids` contient les identifiants uniques des badges qui sont autorisés à débloquer la voiture. Lorsqu'un badge est présenté, le programme vérifiera si son UID est présent dans cette liste pour déterminer si l'accès doit être accordé ou refusé.

## 6. Configuration du servo moteur

```
# Configuration du servo moteur
servo = PWM(Pin(28))
servo.freq(50)

# Fonction pour régler l'angle du servo
def set_servo_angle(angle):
    duty = int(1638 + (angle / 90.0) * 4915)
    servo.duty_u16(duty)
```

Dans cette partie, le servo moteur est connecté à la broche 28 et configuré pour fonctionner à une fréquence de 50 Hz, ce qui correspond à la fréquence standard pour ce type de moteur. Une fonction appelée `set_servo_angle` est définie pour positionner le servo à un angle donné entre 0 et 90 degrés. Le calcul de la valeur PWM est réalisé à partir d'une base de 1638 pour 0 degré et va jusqu'à environ 6553 pour 180 degrés, bien que dans ce projet, l'angle soit limité à 90 degrés. Cette fonction permet ainsi de déplacer mécaniquement un élément, comme un capteur ou une caméra, de manière précise.

## 7. Capteur ultrason pour détection d'obstacle

```
# Configuration du capteur ultrason (TRIG et ECHO)
TRIG = Pin(4, Pin.OUT)
ECHO = Pin(5, Pin.IN)

# Fonction pour mesurer la distance en cm avec l'ultrason
def distance_cm():
    TRIG.low()
    time.sleep_us(2)
    TRIG.high()
    time.sleep_us(10)
    TRIG.low()
    try:
        duration = time_pulse_us(ECHO, 1, 30000)
        return (duration / 2) * 0.0343
    except:
        return 999
```

Le capteur ultrasonique est initialisé avec deux broches : TRIG en sortie pour envoyer le signal et ECHO en entrée pour le recevoir. La fonction `distance_cm` mesure le temps que met l'onde sonore à revenir après avoir rebondi sur un obstacle. Ce temps est ensuite converti en centimètres. Si aucune mesure n'est captée

dans un délai raisonnable, la fonction retourne une valeur élevée (999) pour indiquer qu'aucun obstacle n'est détecté. Ce capteur est fondamental pour le mode automatique, car il permet à la voiture d'éviter les collisions.

## 8. Mouvement oscillatoire du servo

```
# Variables pour faire osciller le servo moteur (balayage)
servo_angle = 0
servo_direction = 1

# Fonction de balayage du servo de 0 à 90 degrés
def balayage_servo():
    global servo_angle, servo_direction
    servo_angle += 5 * servo_direction
    if servo_angle >= 90:
        servo_angle = 90
        servo_direction = -1
    elif servo_angle <= 0:
        servo_angle = 0
        servo_direction = 1
    set_servo_angle(servo_angle)
```

Ce bloc définit un comportement de balayage pour le servo moteur. Il oscille entre 0 et 90 degrés automatiquement en modifiant l'angle par pas de 5 degrés. À chaque extrémité, le sens du mouvement est inversé, créant un va-et-vient continu. Cela peut être utile pour simuler un système de détection visuelle ou pour scanner une zone dans le champ de vision du capteur.

## 9. Attente du badge RFID

```
# État initial affiché sur OLED en attente de badge
oled.fill(0)
oled.text("VOITURE DEBLOQUER ", 20, 0)
oled.text("ID: Waiting", 5, 30)
oled.show()

# Attente de la présentation d'un badge RFID valide
badge_valide = False
led_color(0, 0, 1) # LED bleue allumée

while not badge_valide:
    (stat, tag_type) = rdr.request(rdr.REQIDL)
    if stat == rdr.OK:
        (stat, raw_uid) = rdr.anticoll(0x93)
        if stat == rdr.OK:
            if raw_uid in authorized_uids:
                badge_valide = True
                uart.write("lock:\n")
                led_color(1, 0, 0) # LED rouge
                oled.fill(0)
                oled.text("ID: Access :)", 0, 28)
                oled.text("Card Unlocked", 20, 0)
                oled.show()
                time.sleep(1)
            else:
                led_color(0, 0, 1)
                oled.fill(0)
                oled.text("ID: Not Access", 0, 28)
                oled.text("Card Locked", 20, 0)
                oled.show()
                time.sleep(2)
    oled.fill(0)
```



```

        update_oled_status("MANU", "Left")
    elif direction == "right":
        motorMA.turn_right(value, value)
        update_oled_status("MANU", "Right")
    elif direction == "stop":
        motorMA.stop()
        update_oled_status("MANU", "Stop")
except Exception as e:
    print("Erreur UART:", buffer, "| Exception:", e)
    buffer = ""
else:
    buffer += char

```

La boucle principale commence après la validation du badge. Elle est divisée en deux grandes fonctions. D'un côté, toutes les 50 millisecondes, le capteur ultrason vérifie s'il y a un obstacle. En mode automatique, si rien n'est détecté, la voiture recule ; si un obstacle est détecté, elle s'arrête immédiatement. En mode manuel, l'oscillation du servo continue pour surveiller l'environnement. De l'autre côté, le programme lit en continu les commandes reçues via Bluetooth. Chaque message est analysé pour détecter un changement de mode (manuel ou automatique) ou une direction de déplacement, comme avant, arrière, gauche ou droite. Si l'utilisateur envoie "stop", la voiture s'arrête. Toutes ces actions mettent à jour l'écran OLED pour informer l'utilisateur de l'état courant du système.

## IV.2) Code de la partie commande

```

1 from machine import UART, ADC, Pin, I2C, PWM
2 import time
3 import ssd1306
4
5 # UART et joystick
6 uart = UART(0, baudrate=38400, tx=Pin(0), rx=Pin(1))
7 joystick_y = ADC(Pin(27))
8 joystick_x = ADC(Pin(26))
9 joystick_button = Pin(22, Pin.IN, Pin.PULL_UP)
10
11 # OLED I2C
12 i2c = I2C(0, scl=Pin(21), sda=Pin(20))
13 oled = ssd1306.SSD1306_I2C(128, 64, i2c)
14
15 # Classe pour LED RGB avec PWM
16 class RGBLedPWM:
17     def __init__(self, r_pin, g_pin, b_pin):
18         self.red = PWM(Pin(r_pin))
19         self.green = PWM(Pin(g_pin))
20         self.blue = PWM(Pin(b_pin))
21         self.red.freq(1000)
22         self.green.freq(1000)
23         self.blue.freq(1000)
24
25     def change_led(self, r, g, b):
26         self.red.duty_u16(int(r / 255 * 65535))
27         self.green.duty_u16(int(g / 255 * 65535))
28         self.blue.duty_u16(int(b / 255 * 65535))
29
30 # Initialisation LED RGB PWM
31 led = RGBLedPWM(11, 13, 12)
32
33 # Paramètres
34 dead_zone = 2000
35 center = 32768
36 button_pressed = False
37 auto_mode = False

```

```

38 locked = True
39 last_displayed_dir = ""
40
41 # Affichage OLED
42 def update_display(direction=""):
43     global last_displayed_dir
44     oled.fill(0)
45     if locked:
46         oled.text("== VOITURE BLOQUEE==", 0, 20)
47         oled.text("attente card...", 0, 40)
48         led.change_led(0, 0, 0) # Éteindre LED
49     else:
50         oled.text("Mode: " + ("AUTO" if auto_mode else "MANU"), 0, 0)
51         if not auto_mode:
52             oled.text("SENS: " + direction, 0, 20)
53             led.change_led(0, 0, 255) # BLEU
54         else:
55             oled.text("Auto-control...", 0, 20)
56             led.change_led(0, 255, 0) # VERT
57     oled.show()
58
59 # Affichage initial
60 update_display()
61
62 # Boucle principale
63 while True:
64     val_y = joystick_y.read_u16()
65     val_x = joystick_x.read_u16()
66     direction = ""
67
68     if not locked:
69         if not auto_mode:
70             if abs(val_x - center) > abs(val_y - center):
71                 if val_x > center + dead_zone:
72                     direction = "DROITE"
73                     uart.write(f"right:{val_x}\n")
74                 elif val_x < center - dead_zone:
75                     direction = "GAUCHE"
76                     uart.write(f"GAUCHE:{val_x}\n")
77                 else:
78                     direction = "Stop"
79                     uart.write("stop:0\n")
80             else:
81                 if abs(val_y - center) < dead_zone:
82                     direction = "Stop"
83                     uart.write("stop:0\n")
84                 elif val_y > center + dead_zone:
85                     direction = "EN FACE"
86                     uart.write(f"EN FACE :{val_y}\n")
87                 elif val_y < center - dead_zone:
88                     direction = "EN ARRIERE"
89                     uart.write(f"EN ARRIERE:{val_y}\n")
90             update_display(direction)
91
92     # Changement de mode par le bouton
93     if joystick_button.value() == 0:
94         if not button_pressed and not locked:
95             auto_mode = not auto_mode
96             uart.write(f"auto:{1 if auto_mode else 0}\n")
97             update_display()
98             button_pressed = True
99         else:
100             button_pressed = False
101
102     # Lecture UART (depuis voiture)

```

```

103     if uart.any():
104         try:
105             line = uart.readline().decode().strip()
106             if line.startswith("DEBLOQUEE"):
107                 locked = (line == "DEBLOQUEE:1")
108                 update_display()
109             elif line.startswith("auto:"):
110                 auto_mode = (line == "auto:1")
111                 update_display()
112         except:
113             pass
114
115     time.sleep(0.1)

```

## EXPLICATION DU CODE

### 1. Importation des modules

```

from machine import UART, ADC, Pin, I2C, PWM
import time
import ssd1306

```

Cette première partie importe les modules nécessaires pour accéder aux fonctions matérielles de la carte microcontrôleur. Le module machine fournit des classes pour gérer la communication série UART, les entrées analogiques ADC utilisées ici pour le joystick, les broches numériques Pin, ainsi que les interfaces I2C et PWM. Le module time permet de gérer les temporisations et pauses, tandis que ssd1306 est le pilote pour l'écran OLED utilisé pour afficher des informations à l'utilisateur.

### 2. Configuration de la communication UART et des périphériques joystick

```

# UART et joystick
uart = UART(0, baudrate=38400, tx=Pin(0), rx=Pin(1))
joystick_y = ADC(Pin(27))
joystick_x = ADC(Pin(26))
joystick_button = Pin(22, Pin.IN, Pin.PULL_UP)

```

Ici, la communication UART numéro 0 est configurée avec un débit de 38400 bauds, ce qui permet d'échanger des données entre le microcontrôleur et un autre appareil, typiquement la voiture robotisée. Deux entrées analogiques sont initialisées pour lire les positions verticale et horizontale d'un joystick grâce aux broches 27 et 26. Enfin, un bouton du joystick est configuré en entrée avec une résistance de tirage interne pour détecter ses appuis sans fluctuation parasite.

### 3. Configuration de l'écran OLED via I2C

```

# OLED I2C
i2c = I2C(0, scl=Pin(21), sda=Pin(20))
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

```

Ce segment initialise le bus I2C numéro 0 sur les broches 21 et 20 pour communiquer avec l'écran OLED qui possède une résolution de 128 par 64 pixels. Cela permettra d'afficher des informations textuelles simples à l'utilisateur, comme l'état du système, le mode de conduite ou les directions en cours.

#### 4. Classe de gestion d'une LED RGB avec modulation PWM

```
# Classe pour LED RGB avec PWM
class RGBLedPWM:
    def __init__(self, r_pin, g_pin, b_pin):
        self.red = PWM(Pin(r_pin))
        self.green = PWM(Pin(g_pin))
        self.blue = PWM(Pin(b_pin))
        self.red.freq(1000)
        self.green.freq(1000)
        self.blue.freq(1000)

    def change_led(self, r, g, b):
        self.red.duty_u16(int(r / 255 * 65535))
        self.green.duty_u16(int(g / 255 * 65535))
        self.blue.duty_u16(int(b / 255 * 65535))
```

Cette classe permet de contrôler une LED RGB en utilisant la modulation de largeur d'impulsion (PWM) sur chaque couleur pour ajuster leur intensité lumineuse indépendamment. À la création de l'objet, chaque couleur est associée à une broche en sortie PWM et la fréquence est fixée à 1000 Hz pour un contrôle fluide. La méthode `change_led` prend trois valeurs allant de 0 à 255 représentant l'intensité désirée pour le rouge, le vert et le bleu, et convertit ces valeurs pour régler la puissance effective de chaque LED, ce qui permet d'afficher une large palette de couleurs.

#### 5. Initialisation de la LED RGB PWM

```
# Initialisation LED RGB PWM
led = RGBLedPWM(11, 13, 12)
```

Un objet `led` est créé à partir de la classe précédente en associant les broches 11, 13 et 12 aux composantes rouge, verte et bleue respectivement. Cela prépare la LED RGB pour être contrôlée tout au long du programme, notamment pour donner des retours visuels sur l'état du système.

#### 6. Définition des paramètres et variables d'état

```
# Paramètres
dead_zone = 2000
center = 32768
button_pressed = False
auto_mode = False
locked = True
last_displayed_dir = ""
```

Cette partie définit plusieurs variables importantes pour la logique du programme. Le `dead_zone` correspond à une marge de tolérance autour du centre du joystick dans laquelle aucun mouvement n'est considéré, pour éviter des commandes erratiques dues à de petites variations du joystick. La valeur `center` est le point milieu des lectures analogiques du joystick. La variable `button_pressed` sert à mémoriser l'état du bouton afin d'éviter des rebonds mécaniques lors de son appui. Les booléens `auto_mode` et `locked` indiquent respectivement si le système est en mode automatique ou manuel, et si la voiture est verrouillée ou déverrouillée. Enfin, `last_displayed_dir` mémorise la dernière direction affichée pour éviter des rafraîchissements inutiles.

## 7. Fonction d'affichage sur l'OLED

```
# Affichage OLED
def update_display(direction=""):
    global last_displayed_dir
    oled.fill(0)
    if locked:
        oled.text("== VOITURE BLOQUEE==", 0, 20)
        oled.text("attente card...", 0, 40)
        led.change_led(0, 0, 0) # Éteindre LED
    else:
        oled.text("Mode: " + ("AUTO" if auto_mode else "MANU"), 0, 0)
        if not auto_mode:
            oled.text("SENS: " + direction, 0, 20)
            led.change_led(0, 0, 255) # BLEU
        else:
            oled.text("Auto-control...", 0, 20)
            led.change_led(0, 255, 0) # VERT
    oled.show()
```

Cette fonction actualise l'affichage de l'écran OLED selon l'état du système. Si la voiture est verrouillée, elle affiche un message indiquant ce verrouillage et invite à présenter une carte, tout en éteignant la LED RGB pour signaler l'état inactif. Si la voiture est déverrouillée, elle affiche le mode actif qui peut être automatique ou manuel. En mode manuel, elle affiche la direction prise d'après la position du joystick et allume la LED en bleu. En mode automatique, elle affiche un message indiquant ce mode et allume la LED en vert. L'écran est ensuite rafraîchi pour que ces informations soient visibles immédiatement.

## 8. Affichage initial

```
# Affichage initial
update_display()
```

Juste après la définition de la fonction, l'affichage est initialisé une première fois pour montrer l'état de départ de la voiture, qui est verrouillée et en attente de carte, ce qui correspond à la sécurité du système.

## 9. Boucle principale de contrôle

```
# Boucle principale
while True:
    val_y = joystick_y.read_u16()
    val_x = joystick_x.read_u16()
    direction = ""

    if not locked:
        if not auto_mode:
            if abs(val_x - center) > abs(val_y - center):
                if val_x > center + dead_zone:
                    direction = "DROITE"
                    uart.write(f"right:{val_x}\n")
                elif val_x < center - dead_zone:
                    direction = "GAUCHE"
                    uart.write(f"GAUCHE:{val_x}\n")
            else:
                direction = "Stop"
                uart.write("stop:0\n")
        else:
            if abs(val_y - center) < dead_zone:
                direction = "Stop"
                uart.write("stop:0\n")
            elif val_y > center + dead_zone:
                direction = "EN FACE"
```

```

        uart.write(f"EN FACE :{val_y}\n")
    elif val_y < center - dead_zone:
        direction = "EN ARRIERE"
        uart.write(f"EN ARRIERE:{val_y}\n")
    update_display(direction)

# Changement de mode par le bouton
if joystick_button.value() == 0:
    if not button_pressed and not locked:
        auto_mode = not auto_mode
        uart.write(f"auto:{1 if auto_mode else 0}\n")
        update_display()
        button_pressed = True
else:
    button_pressed = False

# Lecture UART (depuis voiture)
if uart.any():
    try:
        line = uart.readline().decode().strip()
        if line.startswith("DEBLOQUEE"):
            locked = (line == "DEBLOQUEE:1")
            update_display()
        elif line.startswith("auto:"):
            auto_mode = (line == "auto:1")
            update_display()
    except:
        pass

time.sleep(0.1)

```

La boucle principale tourne indéfiniment et commence par lire les valeurs analogiques des axes vertical et horizontal du joystick. Si la voiture est déverrouillée et en mode manuel, elle compare la position relative des deux axes pour déterminer quelle direction a la priorité selon l'inclinaison la plus marquée. Si l'axe horizontal est prédominant et sort de la zone morte, elle envoie une commande Bluetooth indiquant un mouvement à droite ou à gauche selon la position, sinon elle ordonne un arrêt. Si l'axe vertical est plus important, elle envoie une commande pour avancer ou reculer selon la position du joystick. La direction calculée est alors affichée sur l'écran OLED. Ensuite, la boucle surveille le bouton du joystick pour basculer entre mode manuel et automatique, en évitant de multiples déclenchements lors d'un seul appui. Chaque changement de mode est communiqué via Bluetooth et affiché. Enfin, la boucle écoute aussi les messages reçus par UART, notamment les indications de verrouillage ou de mode automatique envoyées par la voiture, et met à jour l'affichage en conséquence. La boucle se termine par une courte pause de 100 millisecondes pour limiter la charge processeur.

### III.3) Conclusion partielle partie logiciel

La phase de programmation a été déterminante dans notre projet, car elle a permis d'assurer l'interaction fluide entre les composants mécaniques et électroniques. Nous avons développé des scripts robustes pour interpréter les signaux des joysticks, gérer la communication Bluetooth via le module HC-05 et afficher les informations sur l'écran OLED. Des algorithmes précis ont été implémentés pour permettre au robot de réagir correctement aux entrées de la manette en mode manuel, et pour naviguer de manière autonome en mode automatique grâce au capteur ultrasonique. Cette étape a renforcé nos compétences en développement embarqué et en intégration de systèmes.

## **V.DIFFICULTES RENCONTREE**

Au cours du développement du projet, nous avons été confrontés à plusieurs défis, tant sur le plan matériel que logiciel.

Sur le plan électronique, l'un des principaux problèmes a été la nécessité de réimprimer notre PCB à deux reprises, en raison d'erreurs détectées après la phase de simulation. Cette correction a engendré des dépenses supplémentaires ainsi qu'un retard dans l'avancement du projet.

Sur le plan mécanique, nous avons rencontré des difficultés dans la conception d'une manette dont le dimensionnement devait être parfaitement adapté à l'intégration de tous les composants (joysticks, LED, module Bluetooth, etc.). Cette exigence de précision a nécessité plusieurs ajustements et itérations sur notre conception 3D.

Du point de vue programmation, la mise en place du projet a exigé des compétences solides en codage embarqué, mais aussi une bonne maîtrise des principes d'électronique de base, notamment pour la réalisation des câblages. Nous avons dû effectuer de nombreux tests et ajustements sur les programmes afin d'identifier les formules adéquates et corriger les erreurs de fonctionnement propres à chaque composant. Le processus de débogage s'est souvent révélé minutieux, des problèmes pouvant survenir à partir de détails mineurs dans le code.

## **VI.CONCLUSION GENERALE**

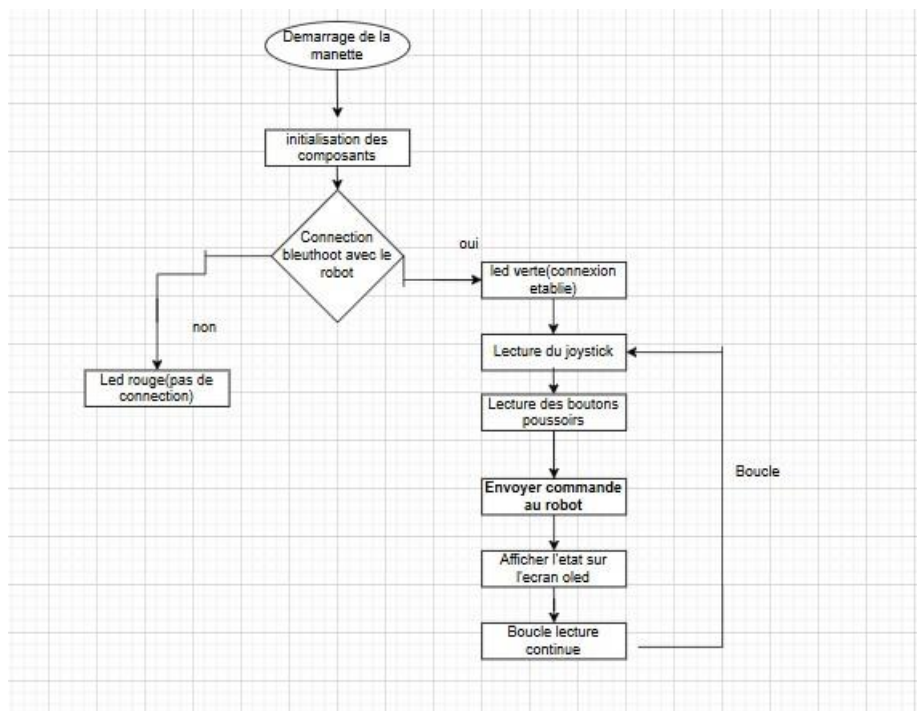
À l'issue de ce projet, nous avons réussi à concevoir et à mettre en œuvre un robot multifonctionnel capable de fonctionner en deux modes : autonome et manuel. Cette double fonctionnalité repose sur une intégration cohérente entre la mécanique, l'électronique et la programmation embarquée. Grâce à l'utilisation de composants comme le module Bluetooth HC-05, le capteur ultrasonique, les joysticks, l'écran OLED et les servomoteurs, nous avons pu répondre à des exigences fonctionnelles variées. Le mode autonome permet au robot d'évoluer de manière intelligente dans un environnement prédéfini, tandis que le mode manuel offre un contrôle direct via une manette conçue sur mesure.

Ce projet nous a permis d'appliquer concrètement les compétences acquises au cours de notre formation, notamment en conception assistée par ordinateur avec Fusion 360, en création de circuits imprimés via JLCPCB, ainsi qu'en programmation et en prototypage électronique. La mise en œuvre du PCB, les nombreux tests de câblage et de compatibilité, ainsi que le débogage logiciel, nous ont permis de mieux comprendre les interactions entre les différentes couches d'un système embarqué. Malgré des difficultés techniques telles que des erreurs de simulation, des ajustements mécaniques précis à effectuer, ou encore des bugs dans le code, nous avons su faire preuve de rigueur et de persévérance pour apporter les corrections nécessaires.

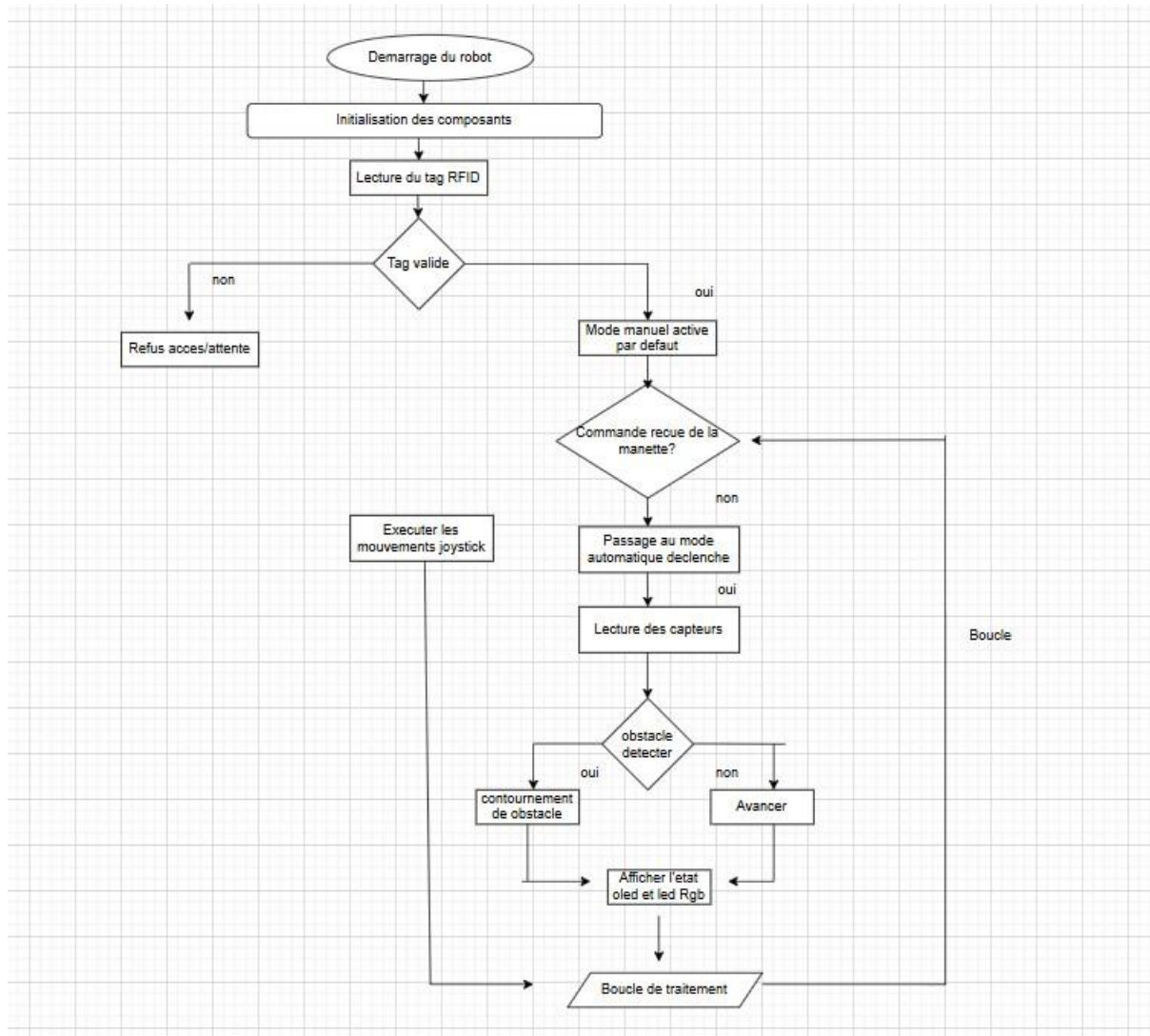
En définitive, ce projet nous a offert une expérience complète et formatrice en automatisation. Il nous a permis de développer notre esprit d'analyse, notre capacité à travailler en équipe et notre autonomie technique, des qualités essentielles pour aborder des projets plus complexes dans le futur.

## VII.ALGORIGRAMME

Pour la partie commande :



Pour la partie mobile :



## VIII.BIBIOGRAPHIE

-Chatgtp :

<https://chatgpt.com/c/682d9970-65ec-800a-abe2-938eea975c82>

-Youtube :

<https://youtu.be/ExHGnBdvz7o?si=Sjx-pKAhWYVAZc3k>

<https://youtu.be/otdLXmWqXWo?si=W50w8svHodAdB0uq>

<https://youtu.be/3KvWVUPXigU?si=A2T8cg3xwJAjLSKF>

-Autre :

[Robot éviteur d'obstacles](#)

<https://www.autodesk.com/be-fr/products/fusion-360/overview?term=1-YEAR&tab=subscription>

## **IX.ANNEXE**

L'ensemble des fichiers sera regroupé dans une archive au format ZIP, puis déposé dans l'espace prévu à cet effet pour la remise du rapport.